

Figure 1: Server screen



Figure 2: Output of netstat

```

23.  cfd=accept(sfd, (struct sockaddr *)&caddr, &len);
24.  if( fork() == 0 ) {
25.      printf("Child Server Created Handling connection
with %s\n",
26.          inet_ntop(AF_INET, &caddr.sin_addr, buff,
sizeof(buff)));
27.      close(sfd);
28.      if(read(cfd, &ch, 1)<0) perror("read");
29.      while( ch != EOF) {
30.          if(((ch>='a' && ch<='z') || (ch>='A' &&
ch<='Z'))
31.             ch^=0x20;
32.          /* EXORING 6th bit will result in change in
case */
33.          if(write(cfd, &ch, 1)<0) perror("write");
34.          if(read(cfd, &ch, 1)<0) perror("read");
35.      }
36.      close(cfd);
37.      return 0;
38.  }
39.  close(cfd);
40. }
41.}

```

Let's see what we've done here. At line 23, after getting the socket descriptor `cfd` from the call to `accept`, we forked the server. The child process (where `pid==0`) closes the listening descriptor with `close(sfd)`, does the work the server is intended to do, and when finished, closes the descriptor and returns (see lines 27-39).

The server, on the other hand, where `pid>0`, just closes the `cfd` (line 39), and is again ready to accept more connections. Thus, for each incoming connection, a new server process will be created. Another method of doing this is using threads, but we're not getting into that right now.

Now, compile, run and see the server running and handling multiple clients. See Figure 1—I'm running

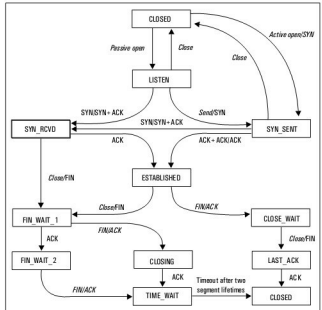


Figure 3: TCP state diagram

the server on a network now ;) though these clients are VirtualBox running VMs with Backtrack (192.168.1.19) and Arch (192.168.1.4), and Android phone running ConnectBot to create a TCP connection. Also run `netstat -a | grep 1205` to check for current network connections; I've grepped 1205, our port, to show only connections to our server (see Figure 2). We can also see the parent server process `LISTEN` ing and the `ESTABLISHED` connections, with IPs and ports.

Making zombies disappear

We added `signal(SIGCHLD, SIG_IGN)` to the code to prevent child processes going into a zombie state. A child process, when it finishes (a clean termination or killed by some signal), returns the exit status to its parent process, which is notified by the `SIGCHLD` signal sent by the system. If the parent doesn't handle this signal, the child process still uses some memory, and remains in a zombie state. If the parent finishes before the child, or doesn't collect the status and terminates, the status will be provided to the parent of all processes, i.e., `init` with `pid 1`. Let's examine some code from the `signal()` man page:

```
#include <signal.h>
```

```
typedef void (*sighandler_t)(int);
```

```
sighandler_t signal(int signum, sighandler_t handler);
```

`signal()` sets the disposition of the signal `signum` to `handler`, which is either `SIG_IGN`, `SIG_DFL`, or the address of a programmer-defined function (a 'signal handler'). It also indicates that the behaviour of `signal()` differs among different Linux and UNIX versions, and tells us to use `sigaction()` instead. The problem is whether the blocked system call, after running the signal handler, will be restarted or not. We can look at this later, when we write our